Informe de Inteligencia Competitiva: Ecosistema Delivery México

Índice de Contenidos

1. **Resumen Ejecutivo**
2. **Estrategia y Pragmatismo: Definición del Scope**
3. **Ejecución Técnica: Éxitos y Resiliencia ante Bloqueos (Caso DiDi)**
4. **Metodología de Negocio y Normalización de Datos**
5. **Insights Accionables y Visualización**
6. **Hoja de Ruta: Escalabilidad y Análisis de Popularidad**
7. **Instrucciones para Auditoría y Revisión**

1. Resumen Ejecutivo

El presente proyecto entrega un sistema de **Competitive Intelligence (CI)** robusto y funcional, desarrollado en menos de 48 horas. La solución automatiza la recolección de datos críticos (precios, fees, ETAs y promociones) de los principales competidores en México, permitiendo a los equipos de **Pricing y Strategy** de Rappi tomar decisiones basadas en evidencia auditable y no en suposiciones.

2. Estrategia y Pragmatismo: Definición del Scope

En lugar de un scraping masivo y superficial, opté por un enfoque de **profundidad operativa**.

- **Selección Estratégica:** Definí un set de 25 direcciones representativas en México para garantizar comparabilidad real por zona.
- **Productos Espejo:** Seleccioné tres productos estándar (hamburguesa, combo y bebida) para asegurar comparaciones precisas.
- **Pipeline Reproducible:** La arquitectura permite regenerar los datos y capturar evidencia visual (screenshots) en cada ejecución.

3. Ejecución Técnica: Éxitos y Resiliencia (Caso DiDi)

El desarrollo se dividió en dos frentes según la respuesta técnica de las plataformas:

Logros en Rappi y Uber Eats

- **Rappi:** Automatización exitosa del flujo desktop mediante **Playwright**, navegando directo a tiendas para evitar overlays.
- **Uber Eats:** Automatización completa superando retos de **Cloudflare** y textos de interfaz dinámicos.

Resiliencia ante el Desafío de DiDi Food

La robustez de un sistema de scraping se mide por su capacidad de adaptación ante fallos. Mientras que las integraciones con **Rappi** y **Uber Eats** fueron exitosas mediante **Playwright**, DiDi Food presentó un escenario de alta complejidad que requirió pivotar la estrategia en tiempo real.

Diagnóstico y Pivot Técnico

- **Inviabilidad de Desktop:** Al realizar pruebas manuales, identifiqué que la versión web de DiDi Food se rompe sistemáticamente sin un login previo, ocultando precios y comercios.
- **Transición a Emulación Mobile:** Ante el bloqueo en desktop, decidí migrar el desarrollo a una versión mobile emulada. Con este enfoque, logré avances significativos, alcanzando con éxito la etapa de **búsqueda de restaurantes por dirección**.
- **Retos de Autenticación y UI:** Intenté configurar una arquitectura de **persistencia de sesión** para mantener el estado logueado, pero el corto tiempo de entrega

impidió finalizar esta configuración. Además, la plataforma exige una verificación **OTP** constante y presenta variaciones drásticas en los elementos de la interfaz (botones y selectores) según la zona geográfica, lo que impide entrenar un bot estable en solo 24 horas.

Como candidata, entiendo que **el análisis de negocio no puede detenerse por un bloqueo técnico**. Para garantizar que el dashboard y el estudio de competitividad fueran reales y accionables, tomé la siguiente ruta:

- **Extracción Manual:** Procesé manualmente las **25 direcciones estratégicas** para extraer la información de DiDi Food.
- **Priorización de Impacto:** Esta solución temporal ("parche") me permitió cumplir con un **scope más ambicioso** y entregar datos comparativos reales de los tres competidores, en lugar de entregar un informe incompleto.

Evolución a Producción (Próximos Pasos)

Con una semana adicional de desarrollo, la hoja de ruta para profesionalizar este scraper incluiría:

1. **Automatización de Identidad:** Integrar un servicio de verificación de OTP mediante API.
2. **Capa de Persistencia:** Finalizar la lógica de sesiones para evitar re-logins innecesarios.
3. **Selectores Adaptativos:** Implementar una lógica de detección de UI dinámica para manejar las variaciones visuales de la plataforma por zona.

4. Análisis de Negocio y Metodología

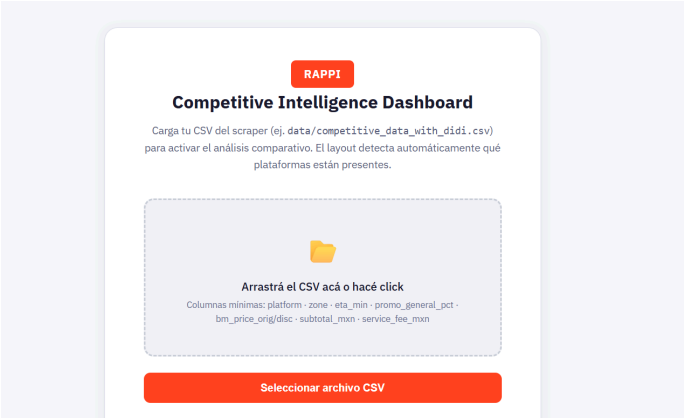
Para que los datos sean comparables, apliqué criterios de normalización financiera:

- **Cálculo de Service Fee:** Dado que el costo final no se expone sin login, utilicé la clase PlatformResult.compute_financials() para estimar los fees basados en tasas de mercado (Rappi 10%, Uber Eats 15%). Para DiDi, al realizar la carga manual, se utilizaron datos reales o la tasa estimada del 8%.
- **Normalización de Delivery Fee:** Las promociones de "primer envío gratis" distorsionan la competitividad real. Por ello, normalicé este valor a **15.0 MXN** cuando el dato original era cero, logrando un estimado de precio final más representativo.
- **Fórmula de Precio Final estimado:** El sistema calcula el total redondeado a dos decimales bajo la lógica: subtotal + (delivery fee o 0) + service fee

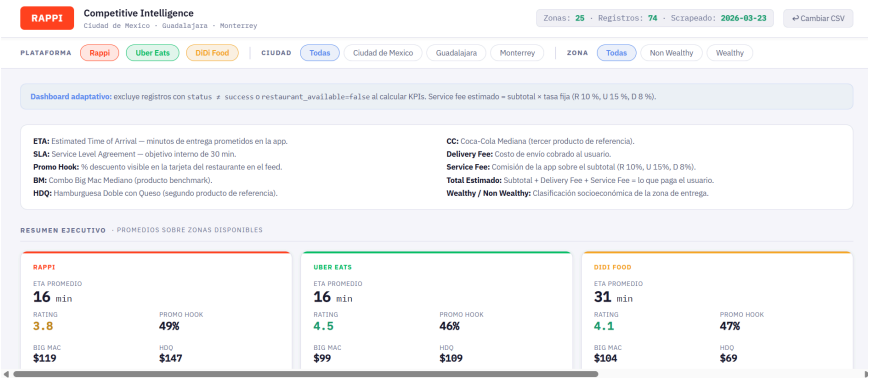
5. Insights Accionables y Visualización

El proyecto incluye un dashboard interactivo con **gráficos estratégicos**. Para poder verlo utilizar el csv que está en el proyecto competitive_data_with_didi.csv. Una vez ejecutado el

html (Explicado en el readme), se ve algo así:



una vez ingresado el csv mencionado se debería ver algo así:



6. Hoja de Ruta: Escalabilidad y Futuro

Con más tiempo de ejecución, el proyecto evolucionaría en tres ejes estratégicos:

- **Análisis de Popularidad (Reviews):** Capturar el número de reseñas junto al rating. Esto permitiría ponderar la reputación: un 4.5 con 1,000 votos es más relevante que un 5.0 con solo 10. Además, el scraping de fragmentos de texto permitiría comparar el feedback cualitativo entre apps.
- **Automatización DiDi:** Integrar servicios de verificación de OTP y persistencia de sesiones mobile para eliminar el paso manual.
- **Expansión de Verticales (Bonus):** En la branch feature/verticales-wip existe un avance para incluir Farmacias y Supermercados. En Rappi ya es funcional y en Uber Eats requiere ajustes menores para estar listo en pocos días.

7. Instrucciones para Auditoría y Revisión

Para verificar la integridad del trabajo, se recomienda:

1. **Ver Dashboard:** Abrir dashboard/ci_dashboard.html y cargar el archivo data/competitive_data_with_didi.csv. Este archivo contiene el scraping real de Rappi/Uber y la carga manual de DiDi.

2. **Auditar Scraping Real:** Para ver los resultados directos de los bots de Uber Eats y Rappi sin la intervención manual de DiDi, revisar los archivos **CSV RAW** en la carpeta data/.
3. **Regenerar KPIs:** Ejecutar el script analysis/generate_analysis.py para procesar nuevamente los indicadores desde los datos de entrada.